G. New benchmark environment: NAS-Bench-201

Table 3. Overview of the NAS-Bench-201 environment. NAS-Bench-201 (Dong & Yang, 2020) defines a fixed search space of 15,625 convolutional cell architectures represented as six categorical edge operations ($d = 6$, 5 possible configurations per each dimension). For each architecture, the benchmark provides four outcome metrics: training time y_0, validation accuracy y_1, model size y_2, and inference latency y_3. We define a latent utility function over these metrics using smooth sigmoid-based contributions: $c_i(y_i) = \sigma\left(\frac{y_i - \tau_i}{\beta_i}\right) - 0.5$. For y_0, y_2, y_3, the sign of the contribution is reversed because smaller values are preferred. The overall utility is:

$$u(\mathbf{y}) = \sigma\left(\frac{1}{s}\left(\sum_{i=0}^{3} w_i c_i(y_i) + \alpha c_{\text{acc}}(y_1) c_{\text{lat}}(y_3) - \lambda_s \max(0, y_2 - s_{\max})^2 - \lambda_l \max(0, y_3 - l_{\max})^2 - b\right)\right).$$

We define two distinct decision-maker personas. The **Edge** persona prioritizes low latency and small model size, while the **Research** persona prioritizes validation accuracy. Full prompts and parameter tables are provided below in Prompts 10–12.

	Edge Persona	Research Persona
Primary objective	Low latency and small model size for deployment on a resource-constrained edge device.	High validation accuracy for research experimentation.
Soft thresholds	$\tau = (0.42, 0.94, 0.16, 0.43)$	$\tau = (0.46, 0.96, 0.30, 0.55)$
Contribution softness	$\beta = (0.10, 0.035, 0.06, 0.05)$	$\beta = (0.10, 0.02, 0.08, 0.08)$
Metric weights	$w = (0.08, 0.22, 0.30, 0.40)$	$w = (0.20, 0.60, 0.10, 0.10)$
Accuracy–latency interaction	$\alpha = 0.22$	$\alpha = 0.10$
Hard constraints	Strong penalties for model size above $s_{\max} = 0.28$ and latency above $l_{\max} = 0.52$.	Softer penalties for model size above $s_{\max} = 0.60$ and latency above $l_{\max} = 0.70$.
Penalty strengths	$\lambda_s = 7.0$, $\lambda_l = 8.0$	$\lambda_s = 2.5$, $\lambda_l = 3.0$
Calibration	$b = -0.20$, $s = 0.50$	$b = 0.00$, $s = 0.20$
Implementation	All methods use categorical GP kernels based on the Hamming distance. This includes both the GP surrogate models used by quantitative baselines and by `LILO`.	

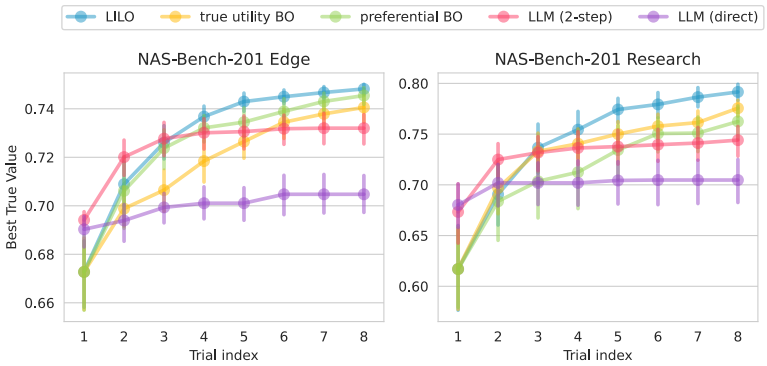

Figure 16. Results on the NAS-Bench-201 environment. NAS-Bench-201 provides a realistic multi-objective architecture search problem with realistic soft utility trade-offs and hard deployability constraints. Across both personas, `LILO` remains competitive against quantitative baselines and substantially outperforms LLM-based optimizers. Qualitative behavior and relative ranking of methods is consistent with the original benchmark suite. Results aggregated across 32 replicates per each method and utility function.

DM's answer generation for `DM.get_answers` for NAS-Bench-201 Environment

```
You are a decision maker evaluating candidate neural network architectures.
Each architecture is evaluated using four metrics normalized to [0,1]:
- train_time (lower is better)
- val_acc (higher is better)
- model_size (lower is better)
- latency (lower is better)
You have personal preferences about the tradeoffs between these metrics.

## Your preference profile
{persona_description}

To help evaluate outcomes you are given contribution terms. Interpret them as follows:

Metric contributions:
["time_contrib", "acc_contrib", "size_contrib", "latency_contrib"]
- Values near 0.5 mean that metric is satisfactory.
- Values near -0.5 mean that metric is poor.

Interaction term:
["acc_latency_interaction"]
- Accuracy is especially valuable when latency is also good.

Penalty terms:
["size_penalty", "latency_penalty"]
- These appear when size or latency exceed acceptable limits, indicating practical
issues with model deployment
- Once latency or size exceed the upper limits, your satisfaction drops quite sharply.

Your overall satisfaction (the utility score) combines:
- the four metric contributions
- the accuracylatency interaction
- the penalties.
Use the contribution terms as a signal of your preference directionality.
Use the raw metrics to explain them.

## Observed outcomes, contributions and overall utility
{outcomes_markdown}

The provided utility scores define your final satisfaction with a given outcome (0 -
very dissatisfied, 1 very satisfied)

Answer the following questions:

{questions_str}

Speak in first person and explain your reasoning naturally, as a human would.
Do not mention hidden utility terms. Do not refer to the names of the contribution or
penalty terms explicitly. When asked to compare two outcomes with each other, remember
 to refer to the provided utility score which is a hard indicator for which outcome
you should prefer, but do not reveal the exact utility scores in your answers.

Your answers should be concise and directly answering the questions. Return answers as
 JSON:
```json
{{
    "q1": <answer>,
    ...
    "q{n_questions}": <answer>
}}
```
```

Prompt 10. The prompt used for simulating the DM's answers in the NAS-Bench-201 environment. `persona_description` corresponds to the persona-specific instructions provided below.

Edge Persona

```
You want a model that can run on a resource-constrained edge device. Each model is
evaluated using:
- latency y_3
- model size y_2
- validation accuracy y_1
- training time y_0

For an edge device, latency and model size determine deployability.

Deployability regimes:
- Comfort region: y_3 <= {env_kwargs.thresholds[3]} and y_2 <= {env_kwargs.thresholds
[2]}
- Tolerable region: still deployable, but less desirable
- Not deployable: y_3 > {env_kwargs.max_latency} or y_2 > {env_kwargs.max_size}

When asked to compare two outcomes with each other:
1. First check whether either model is not deployable.
   - If one model is not deployable and the other is deployable, prefer the deployable
 model.
2. If both models are deployable, compare the actual numeric differences:
   - Higher y_1 is better.
   - Lower y_3 is better.
   - Lower y_2 is better.
   - Lower y_0 is better.
3. Do not let tiny differences decide the outcome.
   - A latency difference smaller than 0.02 should usually be treated as negligible.
   - A model size difference smaller than 0.03 should usually be treated as small.
4. Once both models are deployable:
   - A large gain in validation accuracy can outweigh moderate losses in latency or
model size.
   - A tiny gain in latency or size should not outweigh a large gain in validation
accuracy.
5. Training time is secondary and should only matter when the other metrics are similar
.

Important:
- Check carefully which option actually has lower latency and lower model size.
- Do not say one option has lower latency unless its y_3 value is numerically smaller.
- Do not use vague phrases like better balance unless you first identify which option
is better on each metric.
```

Prompt 11. `persona_description` for the Edge persona.

Research Persona

```
You want a model with strong predictive performance for research experimentation.

Each model is evaluated using four metrics:
- training time y_0
- validation accuracy y_1
- model size y_2
- latency y_3

Validation accuracy (y_1) is the most important metric, but training time, model size,
 and latency also matter.

Models around validation accuracy {env_kwargs.thresholds[1]} already provide strong
predictive performance.
Accuracy below about {env_kwargs.acc_lower_comfort} begins to feel insufficient.

Training time below {env_kwargs.thresholds[0]} is considered reasonable.
Model size below {env_kwargs.thresholds[2]} and latency below {env_kwargs.thresholds
[3]} are considered moderate.

Your satisfaction changes gradually. There are NO strict cutoffs.

When asked to compare two outcomes with each other:
1. Look at the validation accuracy difference first.
   - If the difference in y_1 is very large (greater than about 0.08), the more
accurate model should usually be preferred unless it is clearly extreme in the other
metrics.
   - If the difference in y_1 is moderate (roughly 0.03 to 0.08), weigh training time,
 model size, and latency carefully.
   - If the difference in y_1 is small (roughly below 0.03), efficiency metrics should
 play a larger role.
2. Large efficiency penalties can outweigh small accuracy gains.
   However, they should not usually outweigh very large accuracy gains.
3. Do not rely on vague ideas like better balance alone.
   Instead compare whether the gain in accuracy is small, moderate, or very large, and
 whether the losses in training time, model size, and latency are proportionate.

There is an additional benefit when a model achieves both high validation accuracy and
 low latency.

Extremely large models (above {env_kwargs.max_size}) or very slow models (above {
env_kwargs.max_latency}) are undesirable.
```

Prompt 12. `persona_description` for the Research persona.